ANÁLISE DE SOFTWARE

Arquitetura & Princípios SOLID

Ponto Fácil • diagnóstico e proposta de refatoração

Uma visão sobre responsabilidades, acoplamento e evolução do sistema.



O que é o Ponto Fácil?

Sistema web para apoiar o controle da jornada de trabalho por meio do registro de ponto com reconhecimento facial.

Funcionário

Realiza o registro de ponto e utiliza as funcionalidades disponíveis.

RH / Administração

Gerencia informações e acompanha os registros dos funcionários.

Serviço externo

API de reconhecimento facial integrada ao fluxo do sistema.

Foco da análise: organização interna, dependências e facilidade de manutenção.

Mapeamento com o modelo C4

Nível 1 • Contexto

- Funcionário
- RH
- Ponto Fácil
- API externa de reconhecimento facial

Nível 2 • Contêineres

- Frontend: Vue 3
- Backend: Node.js + Express
- Banco de dados: MySQL

Nível 3 • Componentes

- Routes
- Controllers
- Services
- Models
- Módulos planejados: Time Record, Validation e Facial Service

Atenção: componentes planejados não devem ser confundidos com funcionalidades já implementadas.

O que a análise SOLID revelou

SRP • Principal alerta

Serviços acumulam responsabilidades diferentes.

OCP • Extensibilidade

Mudanças na infraestrutura podem exigir alterações no serviço.

DIP • Dependências

Serviços dependem diretamente de tecnologias concretas.

LSP

Sem violação ativa identificada na análise.

ISP

Oportunidade para definir abstrações menores e específicas.

Objetivo: melhorar a arquitetura existente — não reconstruir o sistema do zero.

SRP: responsabilidades concentradas

Um serviço pode ter mais de um motivo para mudar quando mistura regras de negócio e infraestrutura.

Employee Service

- Regras de negócio do funcionário
- Persistência de dados
- Manipulação / armazenamento de arquivos

→

Employee Service

Regras de negócio do funcionário

Repository

Acesso ao banco

File Storage

Operações com arquivos

Exemplo: trocar o local de armazenamento das fotos não deveria exigir mudanças nas regras de negócio.

OCP e DIP: por que importam?

OCP • Aberto para extensão

Uma nova forma de armazenar arquivos pode exigir modificações no serviço atual.

DIP • Inversão de dependência

Regras de negócio importam diretamente recursos como Sequelize, fs e bcrypt.

Proposta

Introduzir abstrações e injeção de dependências para reduzir o impacto das trocas tecnológicas.

Exemplo: substituir o armazenamento local por nuvem sem reescrever as regras de negócio.

Arquitetura-alvo da refatoração



1. Repository Pattern

Isola consultas ao banco.

2. FileStorage Service

Centraliza operações com arquivos.

3. Injeção de dependências

Facilita trocar implementações.

O que já está bem encaminhado?

Separação em camadas

Controllers recebem requisições e delegam o processamento aos services.

Validação e uploads

Validadores e Multer ajudam a organizar entradas e arquivos.

Estrutura modular

Rotas e módulos separados evitam concentrar tudo em um único componente.

A estratégia é evoluir a base existente e preservar as boas decisões arquiteturais.

Remover vestígios do CivicWatch

O Ponto Fácil utiliza uma aplicação anterior como base. Parte do código herdado pode não representar mais o domínio atual.

Exemplo identificado

A função globalSearch ainda retorna entidades que não pertencem ao domínio do Ponto Fácil.

Ação recomendada

Remover referências obsoletas e alinhar o código às necessidades do controle de ponto.

Menos código irrelevante = menor complexidade e mais clareza para futuras manutenções.

Plano de ação em quatro etapas

01	**Criar EmployeeRepository**	Isolar consultas e persistência.
02	**Extrair FileStorage Service**	Separar arquivos e uploads.
03	**Aplicar injeção de dependências**	Reduzir dependências concretas.
04	**Limpar código herdado**	Remover vestígios do CivicWatch.

Impacto esperado

Testabilidade

Dependências isoladas permitem usar mocks nos testes unitários.

Manutenibilidade

Mudanças na infraestrutura afetam menos as regras de negócio.

Evolução do sistema

Novos módulos podem ser incorporados com menor acoplamento.

Resultado esperado: uma base mais confiável, flexível e simples de evoluir.

CONCLUSÃO

Evoluir a arquitetura, sem recomeçar do zero.

A análise aponta três prioridades:

Separar

Responsabilidades por componente.

Desacoplar

Regras de negócio da infraestrutura.

Simplificar

Remover código fora do domínio.

Obrigado! • Perguntas e discussão